

NBA Statline Forecasting



Arul Selvakumar
DSCI 4350: Data Science Capstone

Introduction

- **Goal**: Predict NBA player statlines with full context prior to game time.
- **Focus**: Build a reproducible ETL forecasting pipeline using historical player and team context.
- **Motivation** : Improve on basic average-based approaches that negate roster and performance form changes.
- **Deliverables** : ETL pipeline, Machine Learning models, evaluation framework, historical replay demo.



arulselvakumar / nba-statline-forecasting



Problem Statement and Project Scope



- NBA player performance highly variable and context-driven.
 - Injuries
 - Rotations
 - Recent Form
- Traditional baseline methods fail to capture these dynamic, nonlinear factors.
 - Inaccurate predictions
- **Goal:** Predict player points, rebounds, and assists using context-aware features.
- **Stakeholders :**
 - Sports betting companies seeking to refine their projection models.
 - Analysts and data scientists concerned with sports prediction models.
 - Teams and media interested in understanding player performance trends.

Project Goals and Methods

Scope :

- Build end-to-end pipeline for NBA statline forecasting.
- Create ML-ready dataset from raw NBA data using feature engineering.
- Compare baseline and advanced regression models across feature sets.
- Evaluate models using time-based validation.

Methodology :

- Linear Regression (linear baseline)
- Random Forest (nonlinear decision trees)
- XGBoost (iterative gradient boosting model)

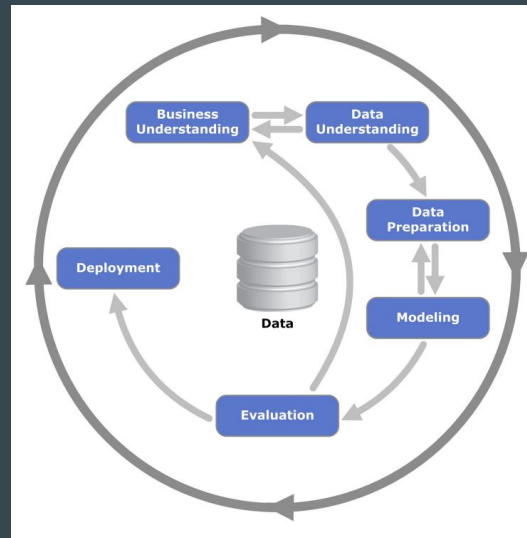
Project Plan and Progress

Checkpoint 1:

- Data collection using *nba_api*.
- Initial EDA to understand player performance patterns.
- Designed structure for ML-ready dataset.

Checkpoint 2 :

- Built full feature engineering pipeline.
- Trained and evaluated numerous ML models.
- Implemented experiment tracking and evaluated error metrics.
- **First Deployment: Historic Replay Demo.**



Dataset Lineage

- **Origin** : Data sourced from **nba_api** for player-level statistics.
 - **Raw Input** : player game logs, season-level usage, opponent metrics
- **Metadata**: player/team identifiers, game date, season, opponent, home/away.
- **Transformations** : season-to-date features, L5/L10 rolling features, teammates-out and missing-production availability features, opponent context features
- **Leakage Rule** : all engineered features designed to only use **available** information prior to a selected game.



swar / nba_api

Feature Engineering

Season-to-Date Features : cumulative averages used as **baseline** performance indicators.

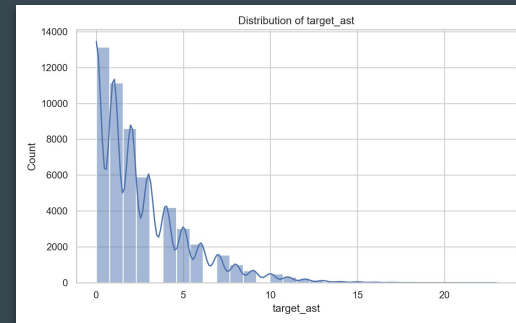
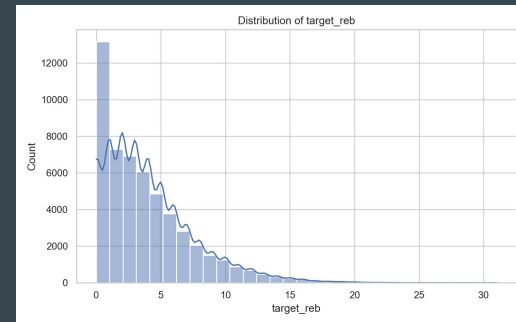
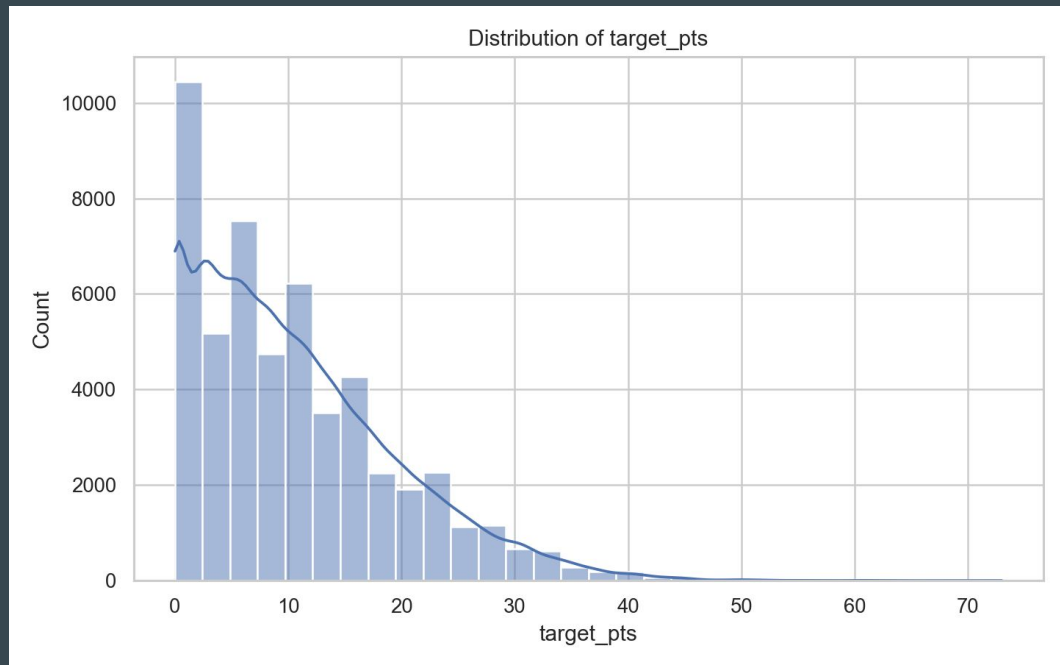
Rolling Features (L5/L10): capture recent player form and shorter-term trends.

Opportunity Features: minutes played used as proxy for player role / usage.

Availability Features : teammate absence indicators determined by prior games.

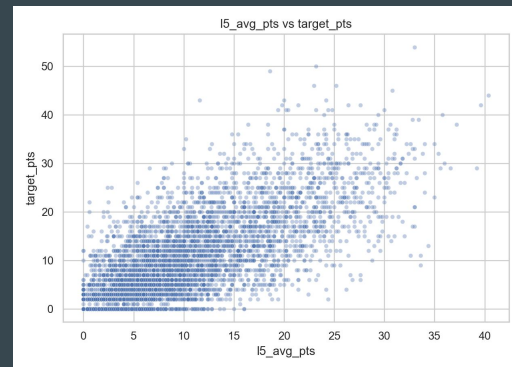
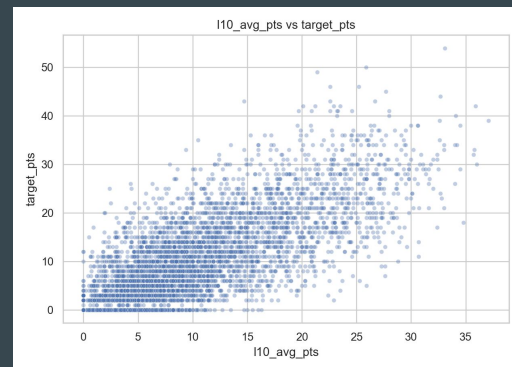
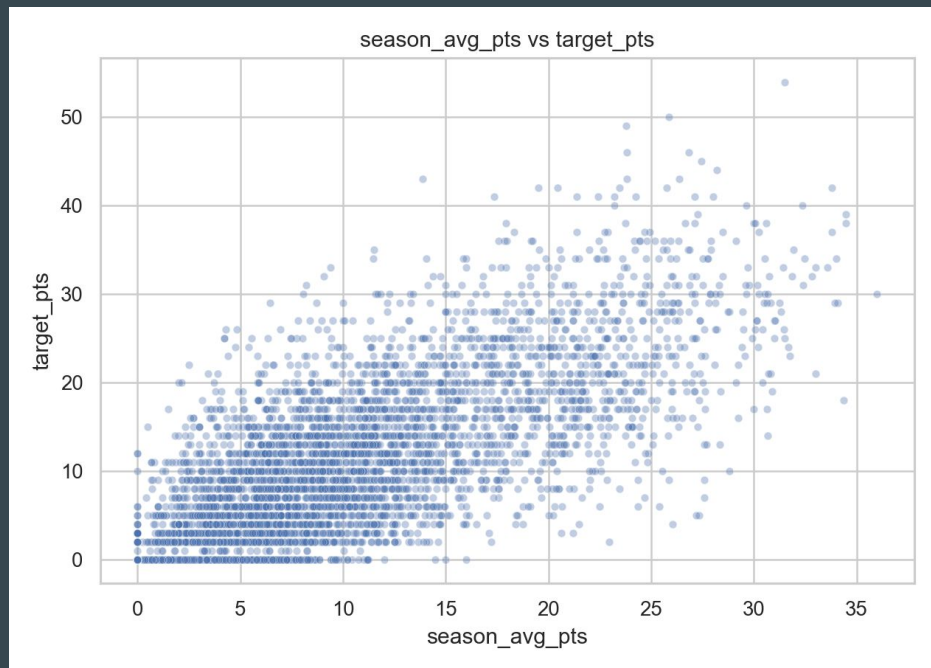
Matchup Context Features : opponent pace + defensive rating to reflect environment.

ML Dataset EDA: Target Distributions



Right-Skewed : most observations in moderate ranges.
Long tails created by occasional high-value performances.

ML Dataset EDA: Feature Relationships



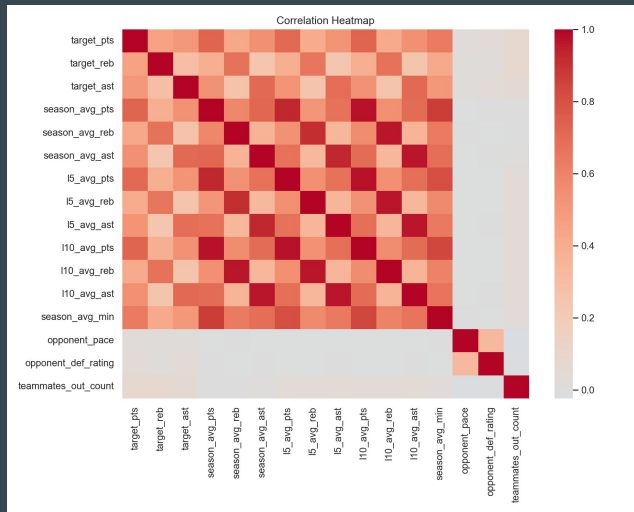
Season Averages : strong baseline relationship.

Rolling Averages : improves short-term predictive signal.

EDA Takeaways

- **Season averages** provide a strong baseline for prediction.
- **Rolling features** improve short-term predictive accuracy.
- **Minutes** (opportunity) are a key-driver of PRA performance.
- **Context features** justify building more advanced models.

Correlation heatmap shows both rolling and minutes played feature relationships.



Modeling: Setup and Evaluation

Separate regression models training individually for points, rebounds, and assists.

Models :

- **Linear Regression** (baseline, linear)
- **Random Forest** (nonlinear)
- **XGBoost** (nonlinear)

Fair assumption that nonlinear models will fit better with such a nonlinear problem.

Evaluation Metrics :

- **MAE** (Mean Absolute Error): average prediction error magnitude
- **RMSE** (Root Mean Squared Error): penalizes larger errors more consequentially

Time-based split used to simulate pregame prediction scenarios.

Modeling: Feature Versioning

Baseline : season-to-date average

v0: adds structured player contexts using season (baseline), L5/L10, opponent, home/away features.

v1: adds availability features - teammate absences and missing production.

v2: adds usage features and richer matchup context (defensive rtg, pace, etc.)

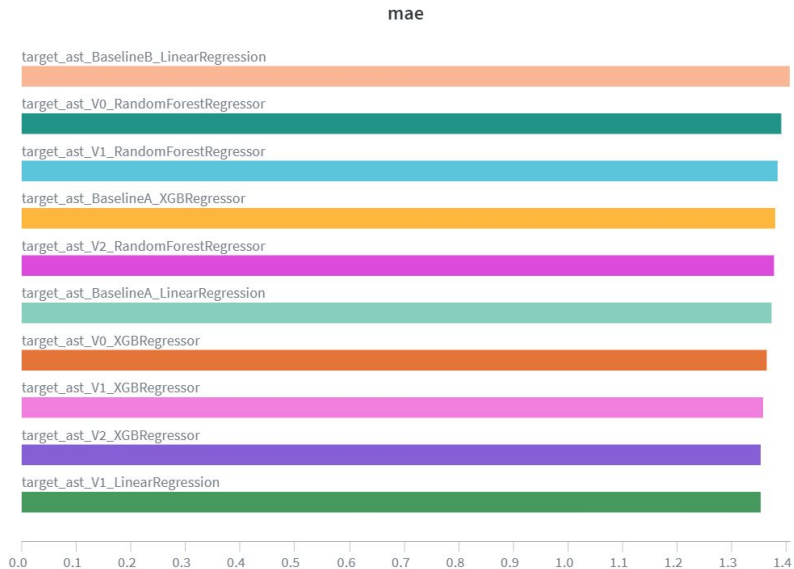
Modeling Results: MAE

Average Prediction Error.

Context-aware models (v0 - v2) consistently outperform baseline.

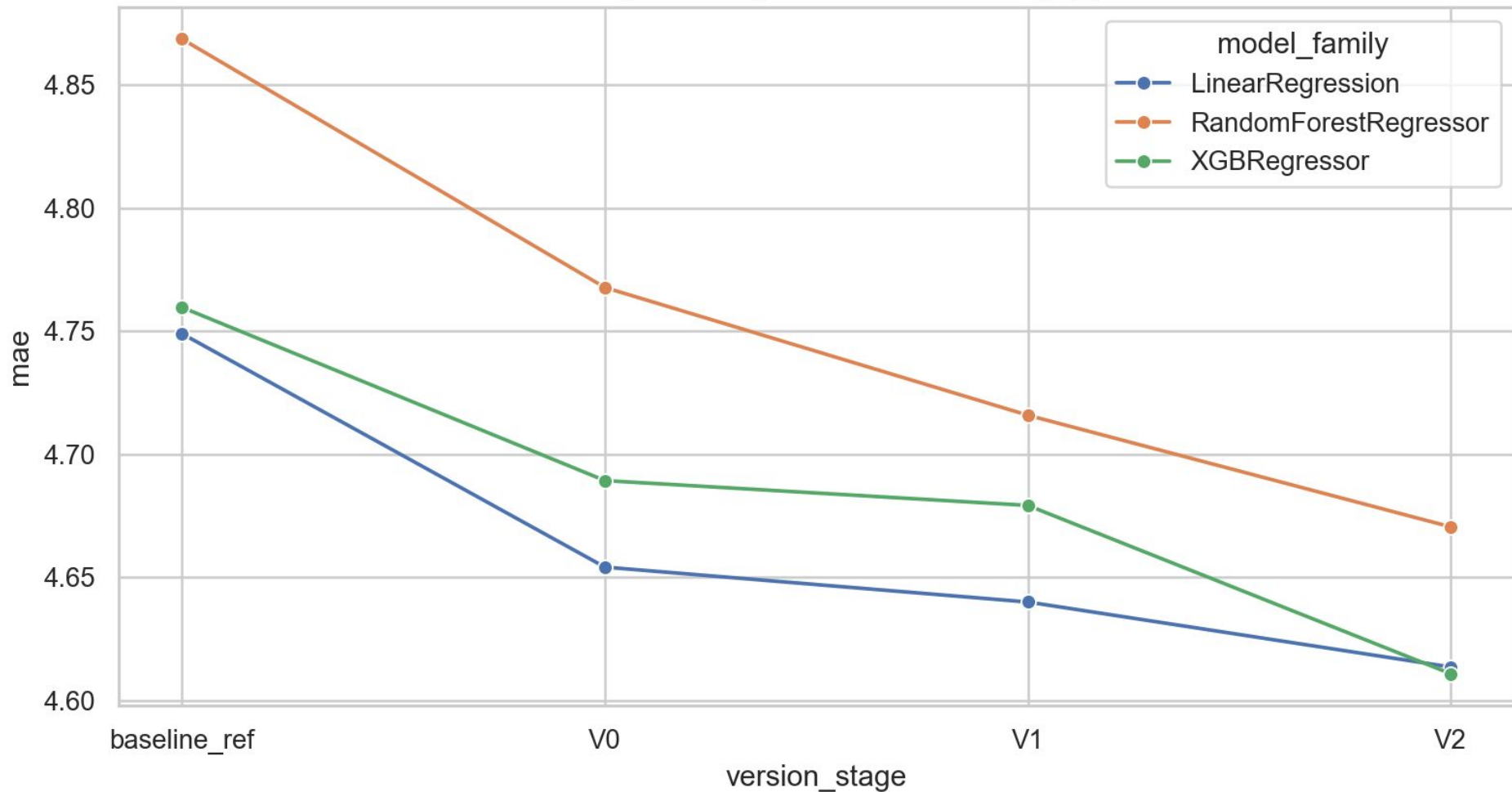
Performance improves as contextual features are added.

Nonlinear models achieve lowest MAE.

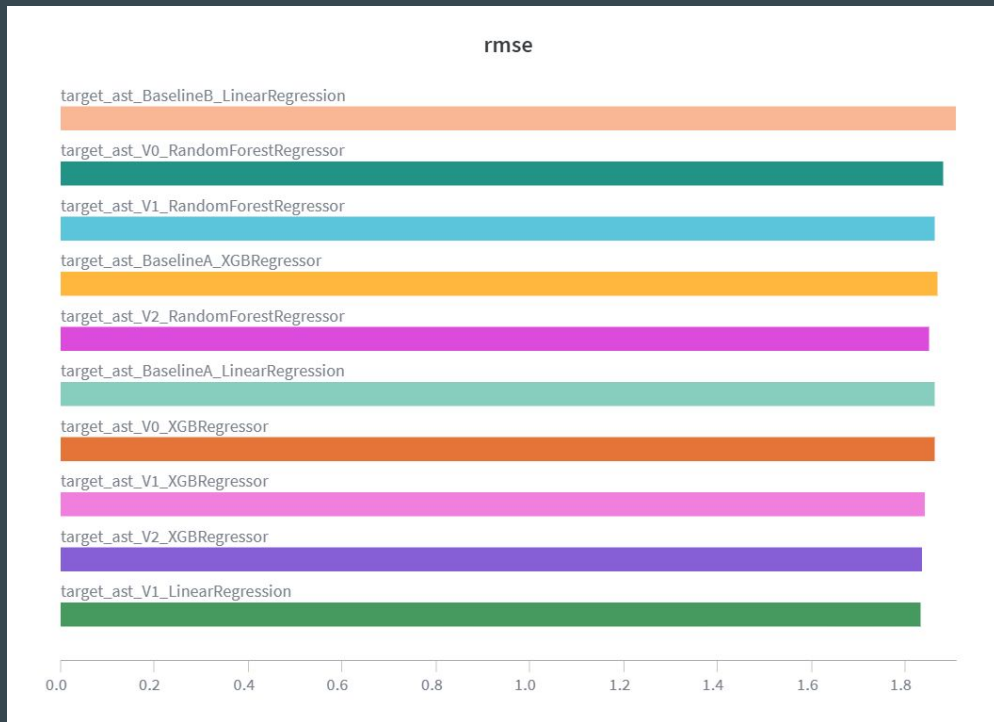


Delete

MAE Progression By Feature Version: target_pts



Modeling Results: RMSE



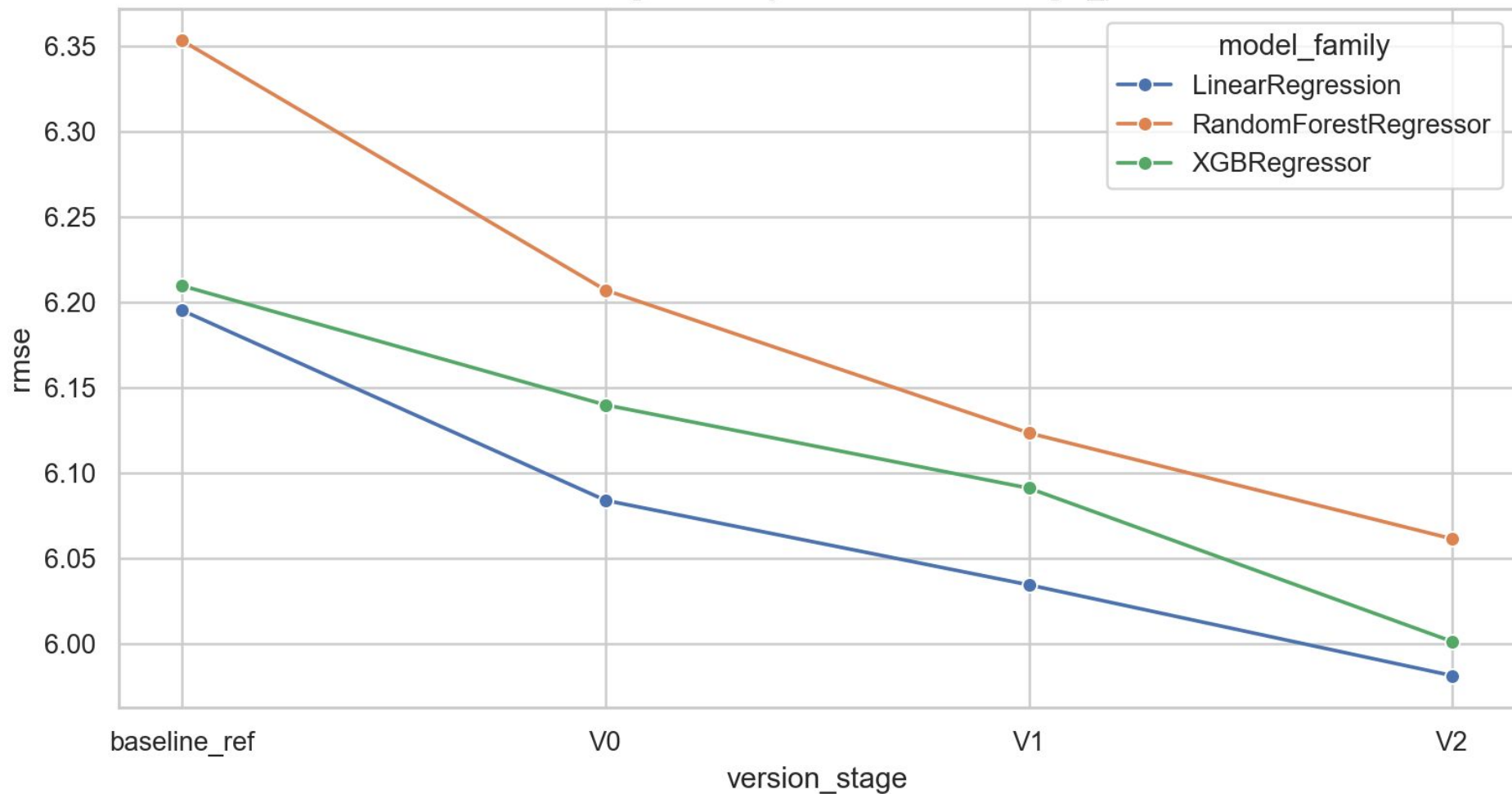
Penalize outlier errors.

Trends consistent with MAE improvements.

Context-aware features reduce large prediction outliers.

Nonlinear models handle variability better than linear models.

RMSE Progression By Feature Version: target_pts



Modeling: Key Takeaways

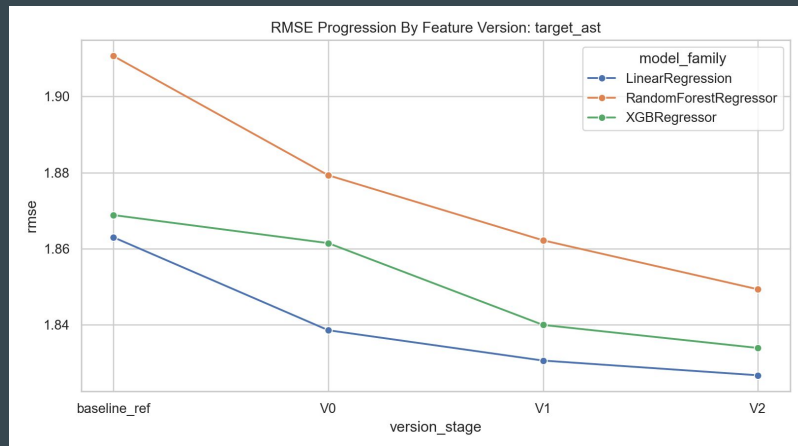
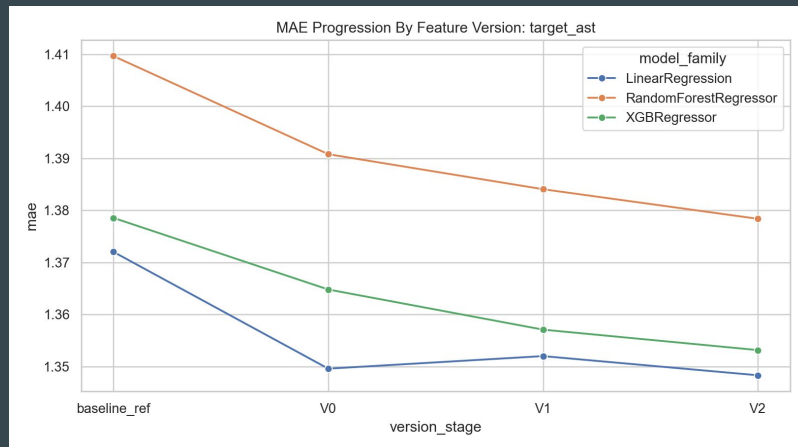
Context-aware feature engineering improves predictive performance.

Incremental improvements: $v0 \rightarrow v2$

Random Forest + XGBoost outperform linear regression and baseline models.

Prediction still remains **noisy**, however improvements are consistent.

**MAE/RMSE for target ast pictured right.*



Experimentation Plan: Validation

Validation Strategy : time-based split using prior seasons for training and future season for evaluation.

- Currently trained on 2023-2024 and 2024-2025 data.
- Isolate pertinent 2025-2026 data from nba.com and *nba_api*.
- **Time-Based:** train the past, evaluate the future (backtesting rolling validation).

Prevents data leakage and reflects pregame prediction scenarios.

Models to be evaluated across same feature versions and model families.

Goal: Minimize MAE / RMSE relative to baseline models.

Experimentation Plan: Optimization and Tracking

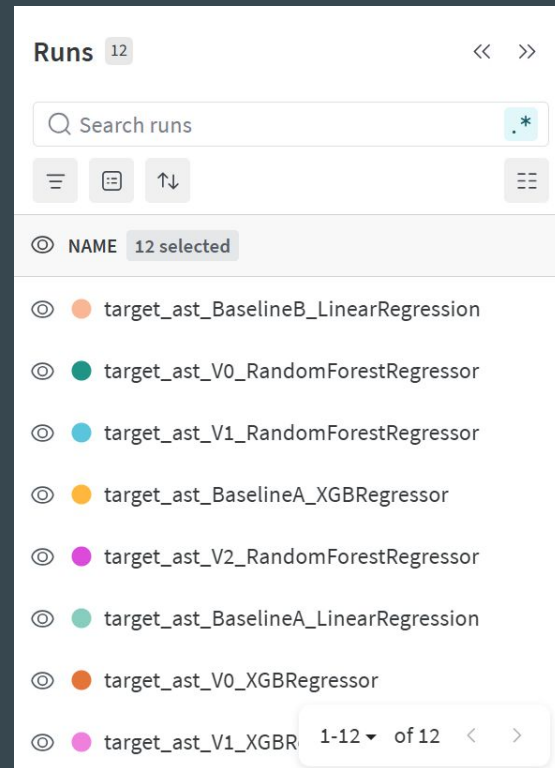
Hyperparameter Optimization :

- Initial models trained on baseline hyperparameters.
- Future work requires hyperparameter tuning.
 - Random Forest: # of trees, max depth, feature selection.
 - XGBoost: learning rate, tree depth, number of boosting rounds.
 - Nonlinear Problem = Nonlinear Solution, explore neural networks.

Weights and Biases (Experiment Tracking) :

- Model Type
- Feature Version
- Evaluation Metrics

Provides a structured framework for reproducibility.



Monitoring & Governance

Model Performance Monitoring :

- Track MAE and RMSE over time on new data (margin ± 0.05).
- Identify degradation in prediction accuracy.

Data Monitoring:

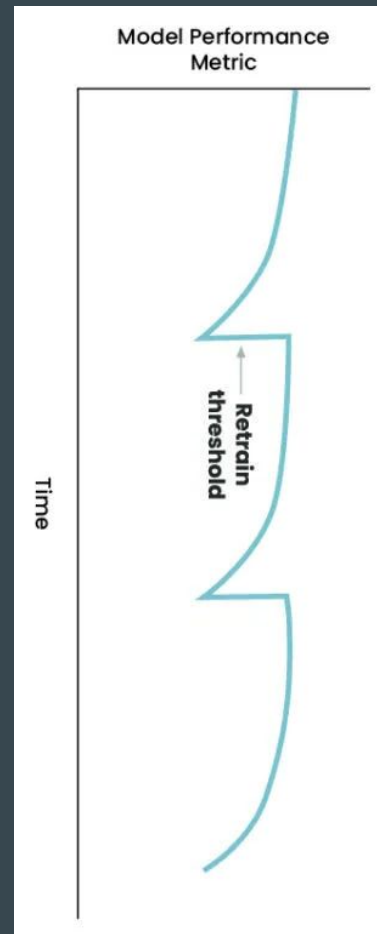
- Monitor changes in feature distributions.
- Detect data drift that may impact model performance.

Re-training Strategy:

- Periodically retrain models using updated season data.
- Trigger retraining when performance drops below a threshold.

Governance :

- Maintain versioned datasets, features, and models.
- All experiments logged on W&B and Github.



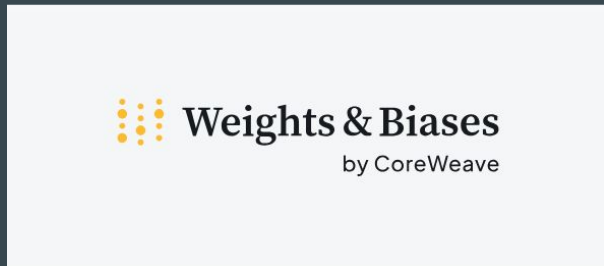
Documentation Plan

Github Repository Documentation :

- Organized Github structure as shown.
- README explains scope, pipeline, and operation.
- Pipeline Documentation:
 - Data Collection (VSCode Jupyter Notebook)
 - Feature Engineering (VSCode Jupyter Notebook)
 - Model Training (VSCode Jupyter Notebook)
 - Evaluation (VSCode Jupyter Notebook)
- Experiment + Metrics Documentation: W&B

Future Extensions:

- Dataset Cards: data sources, features, limitations
- Model Cards: behavior, assumptions, performance



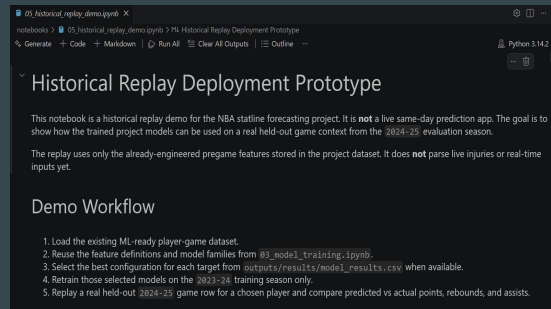
Conclusion and Initial Deployment

Checkpoint Conclusion:

- Constructed complete pipeline for NBA Statline Forecasting from raw data into predictions.
- Context-aware feature engineering improved model performance over versioning.
- Project has thus far shown that incorporating context into NBA prediction leads to measurable improvements.
- Noise undoubtedly exists; regardless consistent gains across feature variations.

Historical replay demo was developed for first efforts of deployment to simulate real pregame prediction scenarios.

- Strong foundation for future real-time application.
- Provided context about individual feature necessities.



... =====
Historical replay for Jayson Tatum on 2024-10-24

...

Historical Replay Output

... This output replays a real held-out game using only the pregame row that existed before the game was played.

...

Game Context

...

player_name	game_date	team	opponent	home_away	season	game_id
Jayson Tatum	2024-10-24	BOS	WAS	away	2024-25	0022400073



Predicted vs Actual Statline

target	model_version	model_family	prediction	actual	absolute_error
Assists	V2	LinearRegression	10.00	6	4.00
Points	V2	XGBRegressor	24.80	25	0.20
Rebounds	V0	LinearRegression	4.56	11	6.44

=====

Historical replay for Nikola Jokić on 2024-11-02

Historical Replay Output

This output replays a real held-out game using only the pregame row that existed before the game was played.

Game Context

player_name	game_date	team	opponent	home_away	season	game_id
Nikola Jokić	2024-11-02	DEN	UTA	home	2024-25	0022400148



Predicted vs Actual Statline

target	model_version	model_family	prediction	actual	absolute_error
Assists	V2	LinearRegression	9.96	9	0.96
Points	V2	XGBRegressor	25.17	27	1.83
Rebounds	V0	LinearRegression	11.01	16	4.99

...=====

Historical replay for Anthony Edwards on 2024-10-26

... Historical Replay Output

... This output replays a real held-out game using only the pregame row that existed before the game was played.

... Game Context

...

player_name	game_date	team	opponent	home_away	season	game_id
Anthony Edwards	2024-10-26	MIN	TOR	home	2024-25	0022400093



... Predicted vs Actual Statline

...

target	model_version	model_family	prediction	actual	absolute_error
Assists	V2	LinearRegression	3.76	4	0.24
Points	V2	XGBRegressor	23.47	24	0.53
Rebounds	V0	LinearRegression	6.10	6	0.10